

# Modelo Predictivo de Turismo Internacional — México

## Random Forest

---

### Objetivo del proyecto

Predecir el flujo mensual de turistas extranjeros que arriban a México por aeropuerto, país de residencia y género. Esta predicción permite a secretarías de turismo y cadenas hoteleras planificar capacidad, campañas y presupuesto con anticipación.

Los datos provienen del sistema **DATATUR** de la Secretaría de Turismo de México y cubren el período 2012–2025.

---

### Algoritmo: Random Forest

Random Forest es un método de aprendizaje supervisado que construye cientos de árboles de decisión en paralelo. Para evitar que todos los árboles aprendan lo mismo, cada uno recibe una muestra aleatoria distinta de los datos y solo puede usar un subconjunto aleatorio de variables en cada decisión. La predicción final es el promedio de todos los árboles individuales.

Esta diversidad forzada hace que el modelo sea robusto: cuando un árbol se equivoca, los demás lo compensan. Es especialmente efectivo en datos heterogéneos como los de este proyecto, que combina 66 aeropuertos con patrones estacionales muy distintos entre sí.

---

### Estrategia de entrenamiento y evaluación

El modelo se entrena con datos históricos de 2012 a 2022 y se evalúa sobre 2023, 2024 y 2025 — años que nunca fueron vistos durante el entrenamiento. Esto simula el escenario real: un modelo entrenado hoy que se usa durante los próximos años sin reentrenamiento.

|               |   |      |   |     |   |      |   |      |  |       |  |       |  |       |
|---------------|---|------|---|-----|---|------|---|------|--|-------|--|-------|--|-------|
| 2012          | — | 2013 | — | ... | — | 2019 | — | 2022 |  | 2023  |  | 2024  |  | 2025  |
| ENTRENAMIENTO |   |      |   |     |   |      |   |      |  | VAL 1 |  | VAL 2 |  | VAL 3 |

Los años 2020 y 2021 fueron excluidos durante la limpieza de datos porque la pandemia de COVID-19 generó patrones completamente atípicos que distorsionarían el aprendizaje del modelo.

---

### Cómo interpretar el MAPE negocio

El **MAPE global** incluye todos los registros del dataset. El problema es que la mayoría de combinaciones aeropuerto + país + mes tienen muy pocos turistas — por ejemplo, 3 turistas de Eslovaquia en el aeropuerto de Acapulco en febrero. Si el modelo predice 5, el error porcentual es 67%. Ese error es

matemáticamente grande pero operativamente irrelevante: ningún hotel toma decisiones basándose en esos segmentos.

El **MAPE negocio** filtra ese ruido calculando el error solo sobre registros que superan un umbral mínimo de turistas. Se calculan cuatro umbrales para ver cómo evolúa la precisión conforme se enfoca en segmentos más relevantes:

| Umbral             | Interpretación                      |
|--------------------|-------------------------------------|
| ≥ 250 turistas     | Actividad mínima para planificación |
| ≥ 1,000 turistas   | Rutas comercialmente activas        |
| ≥ 10,000 turistas  | Destinos de alta demanda            |
| ≥ 100,000 turistas | Segmentos premium en temporada alta |

## 1. Configuración del entorno

Se importan las bibliotecas necesarias y se verifica cuántos núcleos de CPU están disponibles. Random Forest puede entrenar múltiples árboles en paralelo, por lo que usa todos los núcleos disponibles para reducir el tiempo de entrenamiento.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import pickle, json, time, os
import multiprocessing
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import time

plt.rcParams['figure.figsize'] = (13, 5)
plt.rcParams['axes.spines.top'] = False
plt.rcParams['axes.spines.right'] = False

N_CORES = multiprocessing.cpu_count()
print(f'sklearn : {__import__("sklearn").__version__}')
print(f'pandas : {pd.__version__}')
print(f'Núcleos CPU disponibles: {N_CORES}')
print(f'El modelo usará todos los núcleos (n_jobs=-1)')
```

```
sklearn : 1.8.0
pandas : 3.0.2
Núcleos CPU disponibles: 16
El modelo usará todos los núcleos (n_jobs=-1)
```

## 2. Carga del dataset

El dataset ya fue limpiado y procesado en etapas anteriores. Cada fila representa una combinación única de:

- **Aeropuerto:** punto de entrada al país
- **País:** lugar de residencia del turista
- **Género:** hombre o mujer

- **Mes y año:** cuándo ocurrió el flujo

La columna `Valor_Residencia` es el número de turistas de esa combinación en ese mes — lo que el modelo aprenderá a predecir.

Las columnas `Lag_1_mes`, `Lag_12_meses` y `Rolling_3m` son rezagos temporales pre-calculados: representan cuántos turistas llegaron en el mes anterior, en el mismo mes del año pasado, y el promedio de los últimos tres meses.

```
In [2]: URL = 'https://raw.githubusercontent.com/Montiel-Oscar/predictive-tourism-intelligence-mexico/re

print('Cargando dataset...')
df = pd.read_csv(URL)
if 'Año' in df.columns:
    df = df.rename(columns={'Año': 'Ano'})

TARGET = 'Valor_Residencia'
MESES = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun', 'Jul', 'Ago', 'Sep', 'Oct', 'Nov', 'Dic']
UMBRALES = [250, 1000, 10000, 100000]

print(f'Filas cargadas      : {df.shape[0]:,}')
print(f'Columnas           : {df.shape[1]}')
print(f'Años disponibles    : {sorted(df["Ano"].unique())}')
print(f'Aeropuertos únicos  : {df["Aeropuerto"].nunique()}')
print(f'Países de origen    : {df["Pais"].nunique()}')
print(f'\nMuestra de los datos:')
df.head(3)
```

Cargando dataset...

Filas cargadas : 371,011

Columnas : 23

Años disponibles : [np.int64(2012), np.int64(2013), np.int64(2014), np.int64(2015), np.int64(2016), np.int64(2017), np.int64(2018), np.int64(2019), np.int64(2022), np.int64(2023), np.int64(2024), np.int64(2025)]

Aeropuertos únicos : 66

Países de origen : 243

Muestra de los datos:

Out[2]:

|   | Ano  | MesNum | Aeropuerto                                            | Pais    | Región | Sexo   | Valor_Residencia | Valor_Nacionalidad | nac_imput |
|---|------|--------|-------------------------------------------------------|---------|--------|--------|------------------|--------------------|-----------|
| 0 | 2024 | 5      | a.i tulum<br>felipe<br>carrillo<br>puerto, q.<br>roo. | albania | europa | hombre | 1                | 6.0                |           |
| 1 | 2024 | 6      | a.i tulum<br>felipe<br>carrillo<br>puerto, q.<br>roo. | albania | europa | hombre | 1                | 2.0                |           |
| 2 | 2024 | 8      | a.i tulum<br>felipe<br>carrillo<br>puerto, q.<br>roo. | albania | europa | hombre | 4                | 7.0                |           |

3 rows × 23 columns

### 3. Ingeniería de variables

El dataset base incluye variables de rezago temporal, pero se construyen tres grupos adicionales que enriquecen la señal predictiva.

**Estacionalidad por aeropuerto** ( `mes_share_aeropuerto` ): Para cada aeropuerto se calcula qué porcentaje de su tráfico anual llega en cada mes. Por ejemplo, Huatulco recibe el 22% de sus turistas en enero pero menos del 1% en septiembre. Esta variable le da al modelo contexto estacional estructural que los rezagos no pueden capturar directamente. Se calcula *solo sobre datos anteriores a julio de 2022* para no filtrar información del futuro hacia el entrenamiento.

**Rezagos en escala logarítmica** ( `log_lag1` , `log_rolling` , `log_lag12` ): El flujo de turistas varía entre 1 y 300,000 según el segmento. El modelo se entrena sobre el logaritmo del target (para manejar esa distribución asimétrica), por lo que tener los rezagos en la misma escala logarítmica facilita que aprenda las relaciones multiplicativas de forma directa.

**Ratios de tendencia:** `ratio_lag1_lag12` compara el mes anterior con el mismo mes del año pasado — si es mayor a 1, el turismo está creciendo año contra año. `tendencia_reciente` compara el mes anterior con el promedio de los últimos tres meses — detecta si hay aceleración o desaceleración en el corto plazo. Ambos ratios se acotan entre 0.2 y 5.0 para evitar que valores extremos distorsionen el aprendizaje.

In [3]:

```
# Share mensual histórico por aeropuerto
# Solo se calculan datos hasta junio 2022 para evitar filtración temporal
df_share = df[
    (df['Ano'] < 2022) | ((df['Ano'] == 2022) & (df['MesNum'] <= 6))
].copy()

anual = (
    df_share.groupby(['Aeropuerto', 'Ano'])[TARGET]
    .sum().reset_index()
```





Se definen las 22 variables que el modelo usará para hacer sus predicciones y se dividen los datos en conjuntos estrictamente cronológicos.

El conjunto de entrenamiento incluye todos los años disponibles hasta 2022. Es importante que no haya ningún dato de 2023 en adelante dentro del entrenamiento — de lo contrario el modelo tendría acceso a información del futuro y sus métricas de validación serían artificialmente optimistas.

El `assert` al final verifica automáticamente esta condición.

El target se transforma con logaritmo ( `log1p` ) antes del entrenamiento. Esto es necesario porque la distribución del flujo de turistas es muy asimétrica: la mayoría de registros tiene 1-50 turistas pero algunos alcanzan 300,000. En escala logarítmica el modelo aprende mejor porque los valores extremos no dominan el cálculo del error.

```
In [4]: FEATURES = [
    # Variables que capturan la posición en el calendario
    'MesNum', 'mes_sin', 'mes_cos', 'Trimestre',
    'Es_temporada_alta', 'Años_desde_inicio',
    # Estacionalidad histórica del aeropuerto
    'mes_share_aeropuerto',
    # Identificadores geográficos y demográficos (codificados numéricamente)
    'Sexo_enc', 'Region_enc', 'Aeropuerto_enc', 'Pais_enc', 'nac_imputada',
    # Historial reciente en escala original
    'Lag_1_mes', 'Lag_2_mes', 'Lag_3_mes', 'Lag_12_meses', 'Rolling_3m',
    # Historial reciente en escala Logarítmica
    'log_lag1', 'log_rolling', 'log_lag12',
    # Indicadores de tendencia
    'ratio_lag1_lag12', 'tendencia_reciente'
]
FEATURES = [f for f in FEATURES if f in df.columns]

# Conjuntos temporales: entrenamiento hasta 2022, validación en 2023-2025
train = df[df['Ano'] <= 2022].copy()
val23 = df[df['Ano'] == 2023].copy()
val24 = df[df['Ano'] == 2024].copy()
val25 = df[df['Ano'] == 2025].copy()

# Target en escala Logarítmica para el entrenamiento
X_train = train[FEATURES]
y_train = np.log1p(train[TARGET])

# Verificación de integridad temporal
assert train['Ano'].max() == 2022, 'Error: el conjunto de entrenamiento contiene datos de 2023+'
assert val23['Ano'].min() > train['Ano'].max(), 'Error: overlap entre train y val23'

print(f'Variables del modelo      : {len(FEATURES)}')
print(f'\nEntrenamiento (2012-2022): {len(train):,} filas')
print(f'  Años incluidos: {sorted(train["Ano"].unique())}')
print(f'\nValidación 2023 (1 año) : {len(val23):,} filas')
print(f'Validación 2024 (2 años): {len(val24):,} filas')
print(f'Validación 2025 (3 años): {len(val25):,} filas')
print(f'\nIntegridad temporal verificada: sin overlap entre conjuntos.')
```

Variables del modelo : 22

Entrenamiento (2012-2022): 267,799 filas

Años incluidos: [np.int64(2012), np.int64(2013), np.int64(2014), np.int64(2015), np.int64(2016), np.int64(2017), np.int64(2018), np.int64(2019), np.int64(2022)]

Validación 2023 (1 año) : 31,998 filas

Validación 2024 (2 años): 35,460 filas

Validación 2025 (3 años): 35,754 filas

Integridad temporal verificada: sin overlap entre conjuntos.

## 5. Entrenamiento del modelo

Se entrena el modelo con los parámetros definidos.

**2,000 árboles** ( `n_estimators=2000` ): más árboles producen predicciones más estables porque el promedio de más estimadores tiene menor varianza. El modelo converge alrededor de los 1,000-1,500 árboles pero añadir más no perjudica.

**Profundidad máxima 20** ( `max_depth=20` ): cada árbol puede hacer hasta 20 divisiones consecutivas. Una profundidad mayor permitiría memorizar los datos de entrenamiento en lugar de aprender patrones generalizables.

**Mínimo 3 muestras por hoja** ( `min_samples_leaf=3` ): ninguna hoja terminal puede contener menos de 3 observaciones. Esto evita que el árbol cree ramas muy específicas para combinaciones raras que podrían no repetirse en el futuro.

**Raíz cuadrada de variables** ( `max_features='sqrt'` ): en cada división, cada árbol solo evalúa la raíz cuadrada del número total de variables ( $\approx 5$  de 22). Esto fuerza diversidad entre árboles y es la configuración estándar para regresión en RF.

```
In [6]: print(f'Entrenando Random Forest...')
print(f' 2,000 árboles | profundidad 20 | {N_CORES} núcleos en paralelo')
t0 = time.time()

rf = RandomForestRegressor(
    n_estimators = 2000,
    max_depth    = 20,
    min_samples_leaf = 3,
    max_features  = 'sqrt',
    n_jobs        = -1,      # usa todos los núcleos disponibles
    random_state  = 42
)
rf.fit(X_train, y_train)

TIEMPO = round(time.time() - t0, 1)
print(f'\nEntrenamiento completado en {TIEMPO}s ({TIEMPO/60:.1f} min)')
print(f'Árboles: {rf.n_estimators} | Profundidad máxima: {rf.max_depth}')
print(f'Variables por árbol: {rf.max_features} de {rf.n_features_in_}')
```

Entrenando Random Forest...

2,000 árboles | profundidad 20 | 16 núcleos en paralelo

Entrenamiento completado en 48.8s (0.8 min)

Árboles: 2000 | Profundidad máxima: 20

Variables por árbol: sqrt de 22

## 6. Función de evaluación con múltiples umbrales

La función `predecir` toma las variables de entrada, genera la predicción en escala logarítmica y la convierte de vuelta a número de turistas usando la transformación inversa `expm1`. El `np.maximum(..., 0)` garantiza que ninguna predicción sea negativa.

La función `evaluar` calcula cinco métricas:

- **R<sup>2</sup>**: porcentaje de varianza del flujo real que el modelo explica. Un valor de 0.98 significa que el modelo captura el 98% de las fluctuaciones observadas.
- **MAE**: error absoluto promedio en número de turistas. Más interpretable que el RMSE porque no amplifica los errores grandes.
- **RMSE**: similar al MAE pero penaliza más los errores grandes.
- **MAPE global**: error porcentual mediano sobre todos los registros. Está inflado por los segmentos pequeños.
- **MAPE negocio** por umbral: error porcentual mediano sobre los registros que superan cada umbral. Más relevante para el negocio.

```
In [7]: def predecir(X):
        """Predice en escala de turistas reales (revierte la transformación log)."""
        return np.maximum(np.expm1(rf.predict(X)), 0)

def evaluar(y_real, pred, nombre, umbrales=UMBRALES):
    """Calcula y reporta todas las métricas de evaluación."""
    mae = mean_absolute_error(y_real, pred)
    rmse = np.sqrt(mean_squared_error(y_real, pred))
    r2 = r2_score(y_real, pred)
    mape_g = np.median(np.abs((y_real - pred) / np.maximum(y_real, 1)) * 100)

    print(f"\n{'='*62}")
    print(f' {nombre}')
    print(f"{'='*62}")
    print(f' R² = {r2:.4f}')
    print(f' MAE = {mae:>10,.1f} turistas')
    print(f' RMSE = {rmse:>10,.1f} turistas')
    print(f' MAPE global = {mape_g:.1f}% (todos los registros, incl. segmentos pequeños)')

    mapes = {}
    for u in umbrales:
        mask = y_real >= u
        n = mask.sum()
        if n > 0:
            m = np.median(np.abs((y_real[mask] - pred[mask]) / np.maximum(y_real[mask], 1)) * 100)
            estado = 'DENTRO ±15%' if m <= 15 else f'a {m-15:.1f} pts del límite'
            print(f' MAPE ≥{u:>7,} = {m:5.1f}% (n={n:,} registros) {estado}')
            mapes[u] = round(m, 1)
        else:
```

```

print(f' MAPE ≥{u:>7,} = sin registros suficientes')

return {'r2': round(r2,4), 'mae': round(mae,1), 'rmse': round(rmse,1),
        'mape_g': round(mape_g,1), 'mapes': mapes}

# Generar predicciones para cada año de validación
pred23 = predecir(val23[FEATURES])
pred24 = predecir(val24[FEATURES])
pred25 = predecir(val25[FEATURES])

m23 = evaluar(val23[TARGET].values, pred23, 'VALIDACIÓN 2023 – 1 año de distancia')
m24 = evaluar(val24[TARGET].values, pred24, 'VALIDACIÓN 2024 – 2 años de distancia')
m25 = evaluar(val25[TARGET].values, pred25, 'VALIDACIÓN 2025 – 3 años de distancia')

```

```

=====
VALIDACIÓN 2023 – 1 año de distancia
=====

```

```

R²          = 0.9671
MAE          =      94.7 turistas
RMSE         =     1,260.7 turistas
MAPE global  = 37.7% (todos los registros, incl. segmentos pequeños)
MAPE ≥ 250   = 15.5% (n=2,997 registros) a 0.5 pts del límite
MAPE ≥ 1,000 = 13.5% (n=1,589 registros) DENTRO ±15%
MAPE ≥ 10,000 = 12.6% (n=273 registros) DENTRO ±15%
MAPE ≥ 100,000 = 15.7% (n=29 registros) a 0.7 pts del límite

```

```

=====
VALIDACIÓN 2024 – 2 años de distancia
=====

```

```

R²          = 0.9833
MAE          =      77.2 turistas
RMSE         =     926.9 turistas
MAPE global  = 37.7% (todos los registros, incl. segmentos pequeños)
MAPE ≥ 250   = 14.9% (n=3,055 registros) DENTRO ±15%
MAPE ≥ 1,000 = 12.5% (n=1,589 registros) DENTRO ±15%
MAPE ≥ 10,000 = 10.2% (n=270 registros) DENTRO ±15%
MAPE ≥ 100,000 = 6.6% (n=35 registros) DENTRO ±15%

```

```

=====
VALIDACIÓN 2025 – 3 años de distancia
=====

```

```

R²          = 0.9821
MAE          =      78.0 turistas
RMSE         =     973.6 turistas
MAPE global  = 37.3% (todos los registros, incl. segmentos pequeños)
MAPE ≥ 250   = 13.8% (n=2,994 registros) DENTRO ±15%
MAPE ≥ 1,000 = 10.8% (n=1,586 registros) DENTRO ±15%
MAPE ≥ 10,000 = 9.8% (n=265 registros) DENTRO ±15%
MAPE ≥ 100,000 = 6.7% (n=38 registros) DENTRO ±15%

```

## 7. Importancia de variables

Random Forest calcula automáticamente cuánto contribuye cada variable a reducir el error durante el entrenamiento. La importancia de una variable es el promedio de sus contribuciones en todos los árboles del bosque.

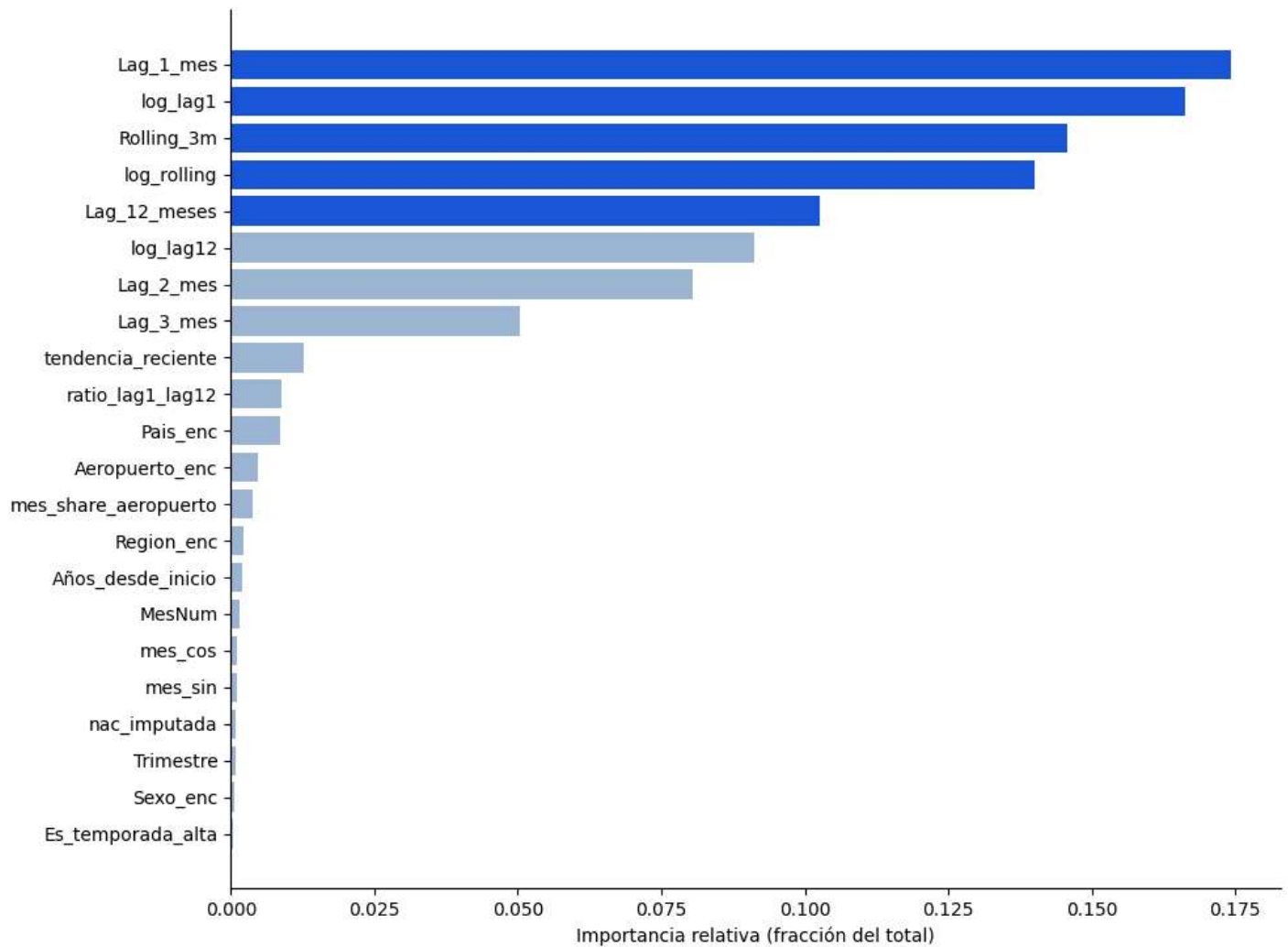
Se espera que los rezagos temporales dominen el ranking: el mejor predictor del turismo del mes siguiente es el turismo de los meses recientes. Las variables de codificación geográfica (aeropuerto, país) aportan la diferenciación entre segmentos.

```
In [8]: importancias = pd.Series(
        rf.feature_importances_, index=FEATURES
    ).sort_values(ascending=False)

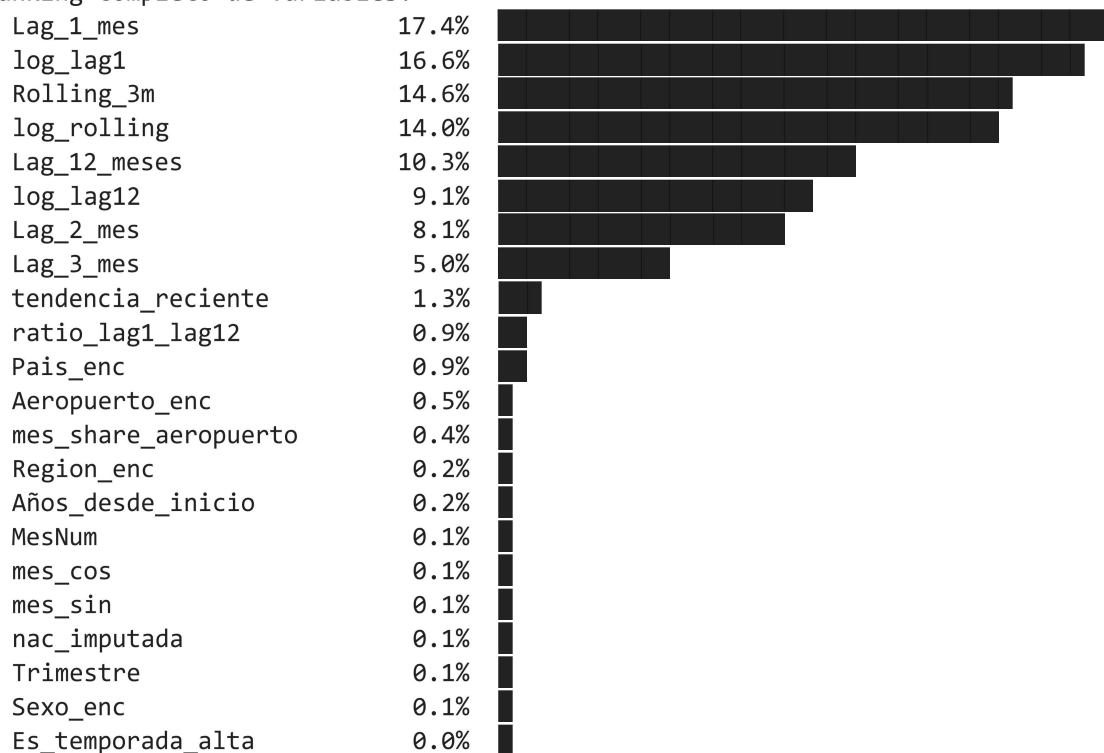
fig, ax = plt.subplots(figsize=(10, 8))
colores = ['#1d58d8' if i < 5 else '#9db4d6' for i in range(len(importancias))]
ax.barh(importancias.index[::-1], importancias.values[::-1],
        color=colores[::-1], edgecolor='none')
ax.set_title(
    'Importancia de variables - Random Forest\n(azul oscuro = top 5 más influyentes)',
    fontsize=13, pad=15
)
ax.set_xlabel('Importancia relativa (fracción del total)')
plt.tight_layout()
plt.savefig('rf_importancia.png', dpi=150, bbox_inches='tight')
plt.show()

print('Ranking completo de variables:')
for f, v in importancias.items():
    barra = '█' * max(1, int(v * 250))
    print(f' {f:<24} {v*100:5.1f}% {barra}')
print(f'\nTop 5 acumulan: {importancias.head(5).sum()*100:.1f}% de la importancia total')
```

# Importancia de variables — Random Forest (azul oscuro = top 5 más influyentes)



## Ranking completo de variables:



Top 5 acumulan: 72.9% de la importancia total



## 8. Diagnóstico visual

Dos gráficas complementarias para entender el comportamiento del modelo:

**Gráfica izquierda — Real vs Predicho (2024):** Si el modelo fuera perfecto, todos los puntos estarían exactamente sobre la línea roja diagonal. La dispersión alrededor de esa línea muestra el margen de error típico. Se omite el 1% superior de valores reales para que la escala sea legible — esos extremos son Cancún en temporada alta.

**Gráfica derecha — Error % por mes (2024):** Muestra en qué meses del año el modelo se equivoca más, calculado solo sobre segmentos con 250 o más turistas. Los meses con mayor error suelen ser temporadas de transición donde los patrones históricos son menos predecibles.

```
In [9]: fig, axes = plt.subplots(1, 2, figsize=(14, 6))

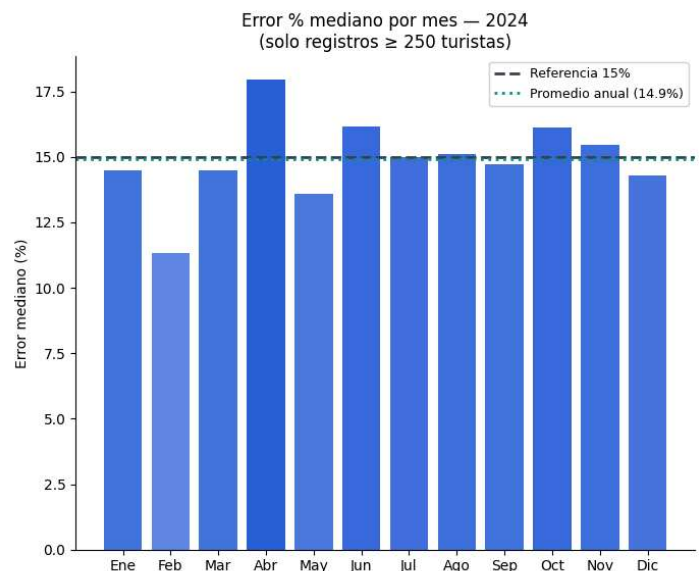
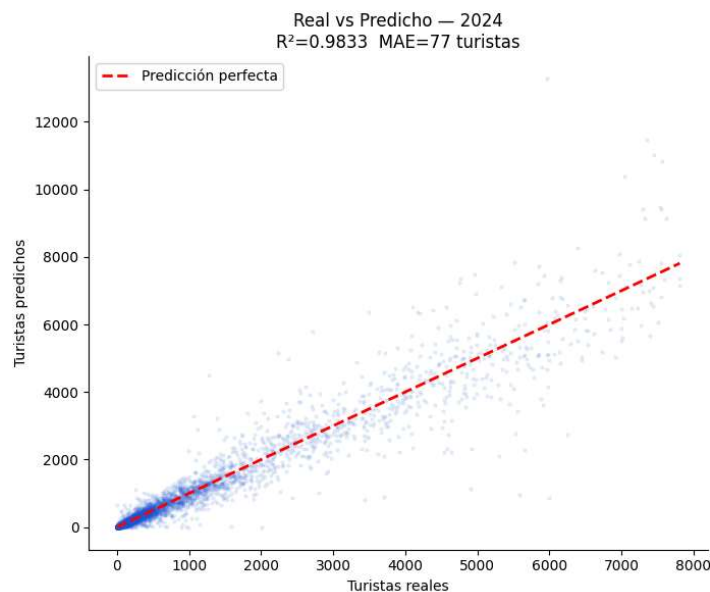
# Gráfica 1: Real vs Predicho
y24 = val24[TARGET].values
p99 = np.percentile(y24, 99)
m99 = y24 <= p99
axes[0].scatter(y24[m99], pred24[m99], alpha=0.08, s=4, color='#1d58d8', rasterized=True)
axes[0].plot([0, p99], [0, p99], 'r--', lw=2, label='Predicción perfecta')
axes[0].set_xlabel('Turistas reales')
axes[0].set_ylabel('Turistas predichos')
axes[0].set_title(f'Real vs Predicho - 2024\nR²={m24["r2"]} MAE={m24["mae"]:.0f} turistas')
axes[0].legend()

# Gráfica 2: Error por mes
v24 = val24.copy()
v24['pred'] = pred24
v24['e_pct'] = abs(v24[TARGET] - v24['pred']) / v24[TARGET].clip(lower=1) * 100
em = v24[v24[TARGET] >= 250].groupby('MesNum')['e_pct'].median()

intens = [0.3 + 0.65*(v/em.max()) for v in em.values]
cols_mes = [[29/255, 88/255, 216/255, a] for a in intens]
axes[1].bar(MESES, em.values, color=cols_mes, edgecolor='none')
axes[1].axhline(y=15, color='#374151', ls='--', lw=2, label='Referencia 15%')
axes[1].axhline(y=em.mean(), color='#0d9488', ls=':', lw=2,
                label=f'Promedio anual ({em.mean():.1f}%)')
axes[1].set_title('Error % mediano por mes - 2024\n(solo registros ≥ 250 turistas)')
axes[1].set_ylabel('Error mediano (%)')
axes[1].legend(fontsize=9)

plt.tight_layout()
plt.savefig('rf_diagnostico.png', dpi=150, bbox_inches='tight')
plt.show()
```





## 9. Demostración — Cancún · EE.UU. · Mujeres

Para ilustrar el comportamiento del modelo en un caso concreto, se muestra la predicción mes a mes para el segmento de mayor volumen del dataset: mujeres residentes en Estados Unidos que arriban al aeropuerto de Cancún.

Este segmento supera los 200,000 turistas en varios meses del año y es el caso de uso central para una cadena hotelera de Cancún que quiera planificar ocupación con 3-6 meses de anticipación.

Las líneas verticales punteadas marcan el cambio de año. La franja sombreada muestra la distancia entre lo real y lo predicho.

```
In [10]: # Combinar los tres años de validación
val_todo = pd.concat([val23, val24, val25]).copy()
pred_todo = np.concatenate([pred23, pred24, pred25])
pred_s = pd.Series(pred_todo, index=val_todo.index)
val_todo['Prediccion'] = pred_s
val_todo['Error_pct'] = (
    abs(val_todo[TARGET] - val_todo['Prediccion']) / val_todo[TARGET] * 100
).round(1)
val_todo['Mes'] = (
    val_todo['MesNum'].map(dict(enumerate(MESES, 1))) +
    ' ' + val_todo['Ano'].astype(str).str[-2:]
)

# Filtrar el segmento específico
cancun = val_todo[
    (val_todo['Aeropuerto'] == 'cancun, q. roo') &
    (val_todo['Pais'] == 'estados unidos') &
    (val_todo['Sexo'] == 'mujer')
].copy().sort_values(['Ano', 'MesNum'])

err_anio = cancun.groupby('Ano')['Error_pct'].mean()
err_str = ' | '.join([f'{a}: {v:.1f}%' for a, v in err_anio.items()])

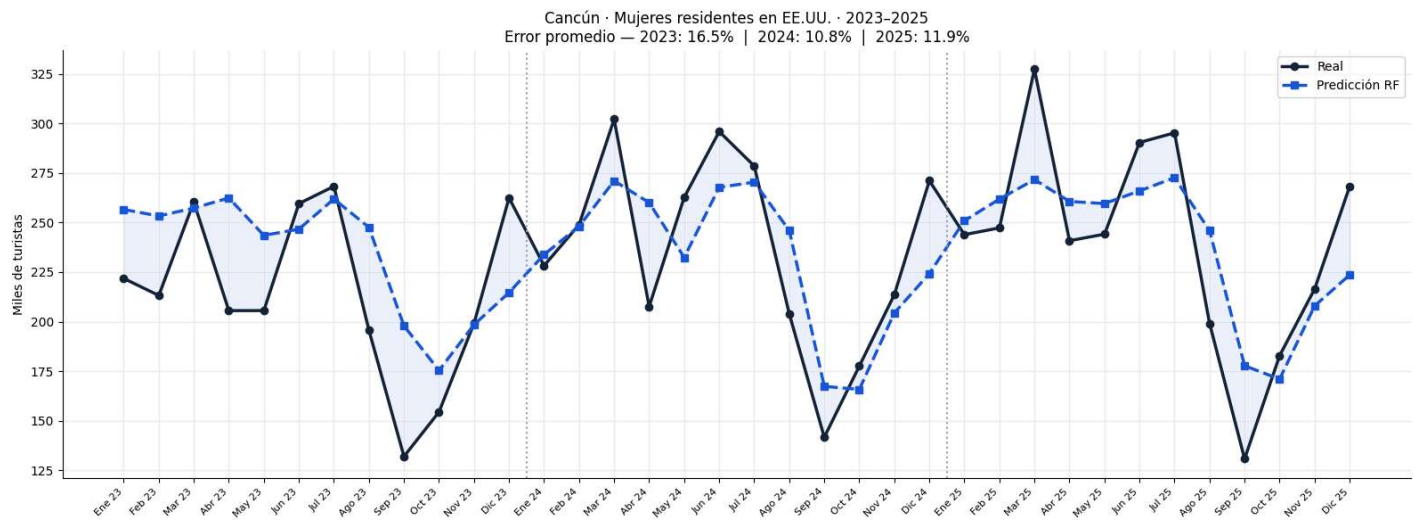
fig, ax = plt.subplots(figsize=(16, 6))
x = range(len(cancun))
```

```

ax.plot(x, cancun[TARGET]/1000, 'o-', color='#14243a', lw=2.5, label='Real', ms=6)
ax.plot(x, cancun['Prediccion']/1000, 's--', color='#1d58d8', lw=2.5, label='Predicción RF', ms=6)
ax.fill_between(x, cancun[TARGET]/1000, cancun['Prediccion']/1000,
               alpha=0.08, color='#1d58d8')
for cx in [12, 24]:
    ax.axvline(x=cx-0.5, color='#9ca3af', ls=':', lw=1.5)
ax.set_xticks(x)
ax.set_xticklabels(cancun['Mes'], rotation=45, ha='right', fontsize=8)
ax.set_ylabel('Miles de turistas')
ax.set_title(
    f'Cancún · Mujeres residentes en EE.UU. · 2023-2025\n'
    f'Error promedio — {err_str}',
    fontsize=12
)
ax.legend()
ax.grid(True, alpha=0.2)
plt.tight_layout()
plt.savefig('rf_demo_cancun.png', dpi=150, bbox_inches='tight')
plt.show()

print('Detalle mes a mes:')
print(cancun[['Mes', TARGET, 'Prediccion', 'Error_pct']].to_string(index=False))

```



Detalle mes a mes:

| Mes    | Valor_Residencia | Prediccion    | Error_pct |
|--------|------------------|---------------|-----------|
| Ene 23 | 221806           | 256563.271843 | 15.7      |
| Feb 23 | 213190           | 253316.234772 | 18.8      |
| Mar 23 | 260371           | 257306.589158 | 1.2       |
| Abr 23 | 205558           | 262398.354500 | 27.7      |
| May 23 | 205572           | 243536.908690 | 18.5      |
| Jun 23 | 259565           | 246658.891087 | 5.0       |
| Jul 23 | 268271           | 261888.233981 | 2.4       |
| Ago 23 | 195684           | 247560.477928 | 26.5      |
| Sep 23 | 131970           | 197881.723544 | 49.9      |
| Oct 23 | 154433           | 175403.965058 | 13.6      |
| Nov 23 | 199131           | 198453.026943 | 0.3       |
| Dic 23 | 262450           | 214673.423500 | 18.2      |
| Ene 24 | 228242           | 233831.435257 | 2.4       |
| Feb 24 | 248937           | 248132.394802 | 0.3       |
| Mar 24 | 302152           | 271005.355497 | 10.3      |
| Abr 24 | 207580           | 260065.283249 | 25.3      |
| May 24 | 262677           | 232398.341634 | 11.5      |
| Jun 24 | 295894           | 267731.997580 | 9.5       |
| Jul 24 | 278541           | 270375.998432 | 2.9       |
| Ago 24 | 203933           | 246034.148908 | 20.6      |
| Sep 24 | 141668           | 167350.491241 | 18.1      |
| Oct 24 | 177578           | 165804.324702 | 6.6       |
| Nov 24 | 213618           | 204362.894358 | 4.3       |
| Dic 24 | 271111           | 224261.204529 | 17.3      |
| Ene 25 | 243891           | 250909.754567 | 2.9       |
| Feb 25 | 247294           | 261825.191781 | 5.9       |
| Mar 25 | 327314           | 271664.391414 | 17.0      |
| Abr 25 | 240831           | 260643.184929 | 8.2       |
| May 25 | 244125           | 259565.731908 | 6.3       |
| Jun 25 | 290404           | 265949.944871 | 8.4       |
| Jul 25 | 295262           | 272627.767539 | 7.7       |
| Ago 25 | 199061           | 245904.391269 | 23.5      |
| Sep 25 | 130766           | 177757.954621 | 35.9      |
| Oct 25 | 182875           | 170986.475377 | 6.5       |
| Nov 25 | 216509           | 207956.027241 | 4.0       |
| Dic 25 | 268268           | 223588.234809 | 16.7      |

## 10. Tabla de resultados consolidada

Resumen de todas las métricas para los tres años de validación. Los valores más relevantes para presentar son los MAPE negocio con umbrales de 1,000 y 10,000 turistas, que reflejan con mayor precisión la utilidad del modelo para decisiones de planificación hotelera.

```
In [11]: print('=' * 82)
print('RESULTADOS - Random Forest')
print(f'Entrenamiento: 2012-2022 | {len(train):,} observaciones | {len(FEATURES)} variables')
print(f'Árboles: 2,000 | max_depth: 20 | {N_CORES} núcleos CPU')
print('=' * 82)

header = f"{'Año de validación':<18} {'R²':>8} {'MAE':>8} {'MAPE global':>12}"
for u in UMBRALES:
    header += f" {'MAPE≥'+str(u):>13}"
print(header)
print('-' * 82)
```

```

for anio, m in [('2023 (1 año)', m23), ('2024 (2 años)', m24), ('2025 (3 años)', m25)]:
    row = f"{anio:<18} {m['r2']:>8} {m['mae']:>8,.1f} {m['mape_g']:>11.1f}%"
    for u in UMBRALES:
        v = m['mapes'].get(u, None)
        row += f" {(str(v)+'%')} if v else 'n/a':>13}"
    print(row)

print('=' * 82)
print(f'\nNota: MAPE global alto (~37%) se explica por los miles de registros con')
print(f'1-10 turistas donde cualquier diferencia produce errores % extremos.')
print(f'El MAPE negocio (≥1,000 turistas) es la métrica relevante para el negocio.')

```

#### =====

#### RESULTADOS – Random Forest

Entrenamiento: 2012-2022 | 267,799 observaciones | 22 variables

Árboles: 2,000 | max\_depth: 20 | 16 núcleos CPU

| Año de validación | R <sup>2</sup> | MAE  | MAPE global | MAPE≥250 | MAPE≥1000 | MAPE≥10000 | MAP |
|-------------------|----------------|------|-------------|----------|-----------|------------|-----|
| E≥100000          |                |      |             |          |           |            |     |
| 2023 (1 año)      | 0.9671         | 94.7 | 37.7%       | 15.5%    | 13.5%     | 12.6%      |     |
| 15.7%             |                |      |             |          |           |            |     |
| 2024 (2 años)     | 0.9833         | 77.2 | 37.7%       | 14.9%    | 12.5%     | 10.2%      |     |
| 6.6%              |                |      |             |          |           |            |     |
| 2025 (3 años)     | 0.9821         | 78.0 | 37.3%       | 13.8%    | 10.8%     | 9.8%       |     |
| 6.7%              |                |      |             |          |           |            |     |

=====

Nota: MAPE global alto (~37%) se explica por los miles de registros con  
1-10 turistas donde cualquier diferencia produce errores % extremos.  
El MAPE negocio (≥1,000 turistas) es la métrica relevante para el negocio.

## 11. Guardar modelo y predicciones

Se guardan cuatro archivos:

- `modelo_rf.pkl` : el modelo completo serializado. Permite hacer predicciones sobre nuevos datos sin reentrenar.
- `predicciones_rf.csv` : las predicciones mes a mes para 2023-2025, útil para el dashboard y el análisis posterior.
- `metricas_rf.json` : métricas de evaluación estructuradas, para comparar con otros modelos.
- Las gráficas generadas durante el análisis.

```

In [12]: import pickle
import json
import os

with open('modelo_rf.pkl', 'wb') as f:
    pickle.dump(rf, f)

pred_out = val_todo[[
    'Ano', 'MesNum', 'Aeropuerto', 'Pais', 'Sexo',
    TARGET, 'Prediccion', 'Error_pct'
]].copy().rename(columns={'Prediccion': 'Prediccion_RF'})
pred_out.to_csv('predicciones_rf.csv', index=False)

```

```

metricas = {
    'modelo': 'Random Forest', 'entrenamiento': '2012-2022',
    'arboles': 2000, 'max_depth': 20, 'features': FEATURES, 'tiempo_s': TIEMPO,
    'r2_2023': m23['r2'], 'mapes_2023': m23['mapes'],
    'r2_2024': m24['r2'], 'mapes_2024': m24['mapes'],
    'r2_2025': m25['r2'], 'mapes_2025': m25['mapes'],
}
with open('metricas_rf.json', 'w') as f:
    json.dump(metricas, f, indent=2)

print('=' * 55)
print('ARCHIVOS GENERADOS LOCALMENTE')
print('=' * 55)
archivos = ['modelo_rf.pkl', 'predicciones_rf.csv', 'metricas_rf.json',
            'rf_importancia.png', 'rf_diagnostico.png', 'rf_demo_cancun.png']

for a in archivos:
    if os.path.exists(a):
        size_kb = os.path.getsize(a) / 1024
        unidad = "KB"
        if size_kb > 1024:
            size_kb /= 1024
            unidad = "MB"
        print(f' {a:<42} {size_kb:>6.1f} {unidad}')
    else:
        print(f' {a:<42} [No encontrado - Revisa si las gráficas se guardaron]')

print(f'\nMAPE negocio ≥ 1,000 turistas:')
for anio, m in [('2023', m23), ('2024', m24), ('2025', m25)]:
    print(f' {anio}: {m["mapes"].get(1000, "n/a")}%' )

```

```

=====
ARCHIVOS GENERADOS LOCALMENTE
=====

```

|                     |           |
|---------------------|-----------|
| modelo_rf.pkl       | 7675.1 MB |
| predicciones_rf.csv | 7.1 MB    |
| metricas_rf.json    | 0.9 KB    |
| rf_importancia.png  | 80.9 KB   |
| rf_diagnostico.png  | 147.6 KB  |
| rf_demo_cancun.png  | 202.0 KB  |

MAPE negocio ≥ 1,000 turistas:

```

2023: 13.5%
2024: 12.5%
2025: 10.8%

```